Playout Control

Table of Contents

- Player URL options
- Using different manifests
- Multiple Server Manifest
- Restart and Catch-up TV options
- Blanking

Player URL options

Some options are available that can be used on a running live stream. Different players/devices may e.g. request a playlist with a different DVR or timeshift window.

All the features specified in Player URLs, Limiting bandwidth use example, Using dynamic track selection or Select variant stream that HLS clients should start playout with are also available for Live.

DVR window length

This feature gives you the ability to dynamically scale your DVR window when needed. The value overrides (and is capped by) the value specified on server manifest creation.

A HTTP Live Streaming playlist becomes very large when using a large DVR window. A typical use case is to use the default value for all streaming formats, and use a lower setting for HLS.

Option	Example link
dvr_window_length=default	default DVR
dvr_window_length=30	No DVR / Pure LIVE
dvr_window_length=300	5 minutes DVR
dvr_window_length=600	10 minutes DVR

Timeshift

Using time_shift you can offset the live point, meaning the player will not start at 'now' but behind the live point. The player will start playing from the archive at the number of seconds specified. Note the player isn't aware of the time shift, so it cannot move forward in time.

This is useful when you are recording content in one timezone and your viewers are in a different timezone.

Option	Example link
time_shift=120	2 minutes time shift
time_shift=600	10 minutes time shift

Using different manifests

Similar to Using different manifests in VOD use cases, different manifests can be very useful for Live playout control.

The difficulty with Live is that the origin updates the server manifest when the encoder starts with the posted audio/video bitrates, so at a minimum these need to be copied.

However, when the bitrates the encoder sends are always the same, then `IsmProxyPass` (Dynamic Manifests) can be used to point to a location returning the manifest to be used - which can contain a selection of the bitrates or extra playout options.

The setup uses two locations in the virtual host, one for the encoder (not using `IsmProxyPass`):

```
<Location />
  UspHandleIsm on
</Location>
```

and one for playout, with `IsmProxyPass`:

```
<Directory "/var/www/live/proxy">
  IsmProxyPass http://live.unified-streaming.com/smil.php/
</Directory>
```

This proxy location then could return a static smil file same as the Live ingest for bitrates (the body part), but with different options in `<head>` part. Notably, HLS AES-128 protected playout is enforced by adding an HLS AES-128 key (while the encoder posts in the clear). PHP could also be used, for instance when parameters need to be injected.

Example

```
<?php
error_reporting(0);
$path_prefix='/var/www/live';
$content_encryption_key="0E9269F650394530DFF0F7CCCEA77B89";
Header('Content-Type: text/xml');
echo "<?xml version=\"1.0\" encoding=\"utf-8\"?>"
?>
<smil xmlns="http://www.w3.org/2001/SMIL20/Language">
  <head>
    <meta name="clientManifestRelativePath" content="channel00.ismc" />
    <meta name="creator" content="Unified Streaming Platform (USP)" />
    <meta name="lookahead_fragments" content="2" />
    <meta name="dvr_window_length" content="1800" />
    <meta name="archive_segment_length" content="3600" />
    <meta name="archiving" content="false" />
    <meta name="restart_on_encoder_reconnect" content="true" />
    <meta name="database_path" content="<?php echo $path_prefix ?>/channel00/channel00.db3" />
    <meta name="iss_playout" content="false" />
    <meta name="hls_playout" content="aes" />
    <meta name="hds_playout" content="false" />
    <meta name="mpd_playout" content="false" />
    <paramGroup id="hls">
      <meta name="key" content=":<?php echo $content_encryption_key ?>" />
      <meta name="license_server_url" content="http://live.unified-streaming.com" />
    </paramGroup>
  </head>
  <body>
    <switch>
      <audio src="file:///<?php echo $path_prefix ?>/channel00/oceans-64k.isma"
systemBitrate="65000">
        <param name="trackID" value="1" valueType="data" />
        <param name="trackName" value="audio" valueType="data" />
        <param name="SamplingRate" value="48000" valueType="data" />
        <param name="Channels" value="2" valueType="data" />
        <param name="BitsPerSample" value="16" valueType="data" />
        <param name="PacketSize" value="4" valueType="data" />
        <param name="AudioTag" value="255" valueType="data" />
        <param name="FourCC" value="AACL" valueType="data" />
        <param name="CodecPrivateData" value="1190" valueType="data" />
      </audio>
      <video src="file:///<?php echo $path_prefix ?>/channel00/oceans-380k.ismv"
systemBitrate="370000">
        <param name="trackID" value="1" valueType="data" />
        <param name="trackName" value="video" valueType="data" />
        <param name="MaxWidth" value="384" valueType="data" />
        <param name="MaxHeight" value="160" valueType="data" />
        <param name="FourCC" value="AVC1" valueType="data" />
        <param name="CodecPrivateData"
value="000000016742C00D96640C0ADFF80C800C88800001F480005DC0078A15500000000168CE32C8"
valueType="data" />
      </video>
    </switch>
  </body>
</smil>
```

It's important to note that absolute paths need to be used for database and file locations. Without these the origin cannot find the db3 and ismv files used for ingest.

The player request (as defined in Player URLs) looks like this:

```
#!/bin/bash

curl -v http://live.unified-streaming.com/proxy/channel00/channel00.isml/manifest
```

The Encoder URL on the other hand looks like this:

```
http://live.unified-streaming.com/channel00/channel00.isml
```

and uses the original location **without** `IsmProxyPass`.

Care must be taken when setting up the publishing point: the location of the .db3 file has to be specified so both ingest and playout share the same database. The path to the .db3 file is specified like this:

```
#!/bin/bash

mp4split --database_path=/var/www/live/channel00/channel00.db3 -o test.isml
```

Without database_path the origin will use the relative path, wrongly prepending the .db3 file with 'smil.php'. The corresponding error will look like this:

```
FMP4_404 /smil.php/channel00/channel00.db3: unable to open database file (sqlite3_open_v2)
```

The above example uses a simple PHP script that returns a smil file, but other locations may be used for IsmProxyPass as well, for instance a CMS or other. Query parameters may be used to trigger selection of options or pass paths to .db3 and .ismv files.

Multiple Server Manifest

It is sometimes convenient or necessary to share ingested content between multiple server manifests. For instance, playout from the primary ingest pubpoint is in the clear and egress access is limited to internal IP addresses, while the same content is offered publicly with DRM. To make this work, the second server manifest should reside in the same directory and point at the database file created by the first (ingest) pubpoint.

1. Create pubpoint with all options relevant to ingest (only) using the HTTP API.
2. Point the encoder at this point and start the ingest. This creates the db3 file and populates the `<body>` of the server manifest.
3. Retrieve the ingest-only server manifest (either from filesystem or using GET through HTTP API) and remove the `<head>` section while preserving `<body>`:

   ```
   sed "\=<head>=,\=</head>=d" ingest.isml > tracks.isml
   ```

4. Create the second server manifest with the options relevant to egress. Make sure *archive segment length* matches the ingest and point `--database` at the ingest db3:

   ```
   mp4split -o egress.isml --database_path=ingest.db3 \
                           --archive_segment_length=... \
                           --dvr_window_length=... \
                           --cpix ... \
                           ... \
                           tracks.isml
   ```

Live CMAF archive

With the experimental --archive_cmaf option (where storage-MPD replaces the database) you can use the REST API to create both ingest and egress pubpoints in a similar fashion. Instead of pointing `--database_path` at the ingest `.db3` file, in this case you should point `--storage_mpd_url` at the `.smpd` file in the ingest pubpoint.

The storage MPD can be accessed locally (using a relative path or `file:///` URL) or over HTTP. The latter allows physical separation of the ingest server from the egress pubpoint, additional HTTP caching layers, replication and scaling horizontally.

Restart and Catch-up TV options

By default the live presentation is made available and the player starts playback at the live edge together with a scrub bar that includes the full length of the DVR window.

But there may be use cases where you would not wish to provide the ability to scrub beyond the programme you are making available.

For example if you are providing an EPG (Electronic Programme Guide) and you are providing a way of entering ongoing programmes that haven't finished yet.

The user may not have access to watch anything before or after this programme. For example the eight o'clock news is broadcast after the movie Guardians of the Galaxy and the user doesn't have the access rights to scrub back before the news and watch it. The same goes for what is being broadcast afterwards, ideally you would like to restrict the end point of the manifest to the end point (in the future) of the news and nothing beyond that.

It is also a common feature of live EPGs that when you enter an ongoing programme you have the option of a 'start over' or 'restart' button, that jumps back to the beginning of the programme.

This feature also brings parity to the link structure you may already be using in a CMS to provide either a reverse EPG or yet to be archived clips in your offering.

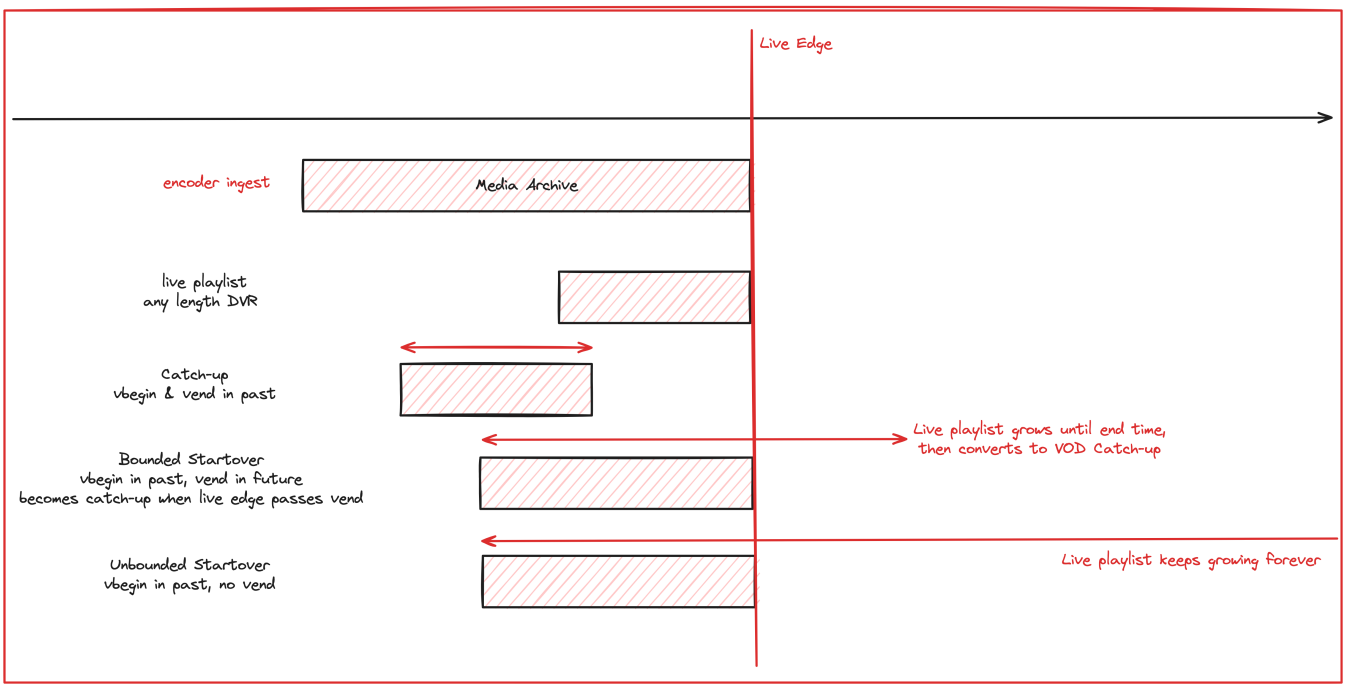
By using the restart and catch-up options all the above functionality can be delivered simply and reliably using time stamps on the URL.

Scenarios and Manifest types:

- Catch-up: if you pass a start time and end time in the past, then the Origin returns a VOD manifest of the program.
- Bounded startover: if you pass a start time and end time in the future, then the Origin returns a manifest that allows you to rewind to the beginning of the program, but automatically ends at the given end time converting to a Catch-up VOD manifest.
- Unbounded startover: if you pass only a start time, then the Origin returns a LIVE manifest which keeps growing but with a fixed start point, allowing rewinding back to that time.

Note

Setting only an end time, i.e. `vend` parameter without `vbegin`, is not supported for live streams.



Setting the time ranges in a request is done by using either the `t` or `vbegin` and `vend` query parameters with ISO 8601 formatted datetime strings. For the `t` parameter the value should be a range separated by `-` as in the examples below. Alternatively you can set the start and end times separately in the `vbegin` and `vend` parameters. For clarity it is generally preferred to use the separate options.

Assuming the current time is 2015-07-28T20:22:00.000 (20:22 on the 28th of July)

Option	Description
t=2015-07-28T20:00:00.000	Start at the beginning of the
t=2015-07-28T20:00:00.000-2015-07-28T20:30:00.000	The complete eight o'clock n
t=2015-07-28T20:00:00.000-2015-07-28T20:20:00.000	The complete eight o'clock n
vbegin=2015-07-28T20:00:00.000&vend=2015-07-28T20:20:00.000	The same as the previous exa

The above scenario is ideal if you have for example a 24 hour rolling DVR window for Live.

Please note while we follow ISO8601 formatting we do not support the use of timezones to offset UTC

If you would like to run a catch-up service with a larger window, say 7-10 days, using Unified Capture would be a much better fit. Using the same time notation and feature set, the live streams can be archived to disk as they become available. Please read more about it here: Capturing LIVE.

> **Note**
>
> Using HLS or DASH (the latter with the default `isoff-live` profile), Restart and Catch-up TV can be used with a start and end time, or a start time only. Using Smooth Streaming, both a start and end time must be supplied.

Blanking

A specific use case of the second scenario in above section is 'Blanking' where the origin makes streams unavailable for a specific time to all or only specific users, for instance because streaming rights do not include certain content or because user A has a premium subscription and user B has not.

For example, a service provider acquires the right to stream certain TV-channels, and gets the rights for sports as well - but sports are only available for a premium subscription.

Say the game starts at 20:30, then when Pippin joins to view at 19:50 he is handed a manifest which is only valid until 20:30. Simon on the other hand has the premium option and gets the full stream.

In the case of Pippin the CMS has to tag begin and end time *in the future* onto his manifest request.

So Pippin's player will request something like this:

```
http://www.theservice.com/channel1/channel1.isml/.mpd?t=2015-07-28T19:50:00.000-2015-07-
28T20:30:00.000
```

Whereas Simon's player will request the plain URL:

```
http://www.theservice.com/channel1/channel1.isml/.mpd
```